Resolution: Unification

Video Lesson

Duration: 00:15:00

Slide Deck

M09S06 (https://canvas.tamu.edu/courses/430127/files/92570678?wrap=1)

Lesson Transcript

Unification

- A substitution θ is called a **unifier** for a set $\{E_1, ..., E_k\}$ iff $E_1\theta = E_2\theta = ... = E_k\theta$.
- The set $\{E_1, ..., E_k\}$ is said to be **unifiable** if there is a unifier for it.
- A unifier σ for a set $\{E_1, ..., E_k\}$ of expressions is a **Most General Unifier** iff for each unifier θ for the set there is a substitution λ such that $\theta = \sigma \circ \lambda$.
- A Most General Unifier will avoid unnecessary substitution(s).

We discussed the concepts of substitution and composition of substitution in the previous lecture. Thus, now, we can finally talk about unification.

A substitution *θ* is called the unifier for a set of expressions, E_1 to E_k, if and only if when we apply this substitution to each of these expressions, then the resulting expression would be equal.

A set of expressions, E_1 to E_k, is said to be unifiable if there is a unifier for it.

In a unifier, *σ*, for a set of expressions, is the most general unifier if and only if for each unifier *θ* for the set, there is a substitution *λ* such that *θ* equals σ, the most general unifier composed with *λ*.

This simply means that the most general unifier will avoid unnecessary substitutions.

Examples: Unification

$P(x, g(x))$ will unify with:

Expression	Necessary Substitution
$P(x, y)$	$\{y/g(x)\}$
$P(z, g(z))$	$\{z/x\}$ or $\{x/z\}$
$P(Socrates, g(Socrates))$	$\{x/Socrates\}$
$P(x, g(y))$	$\{x/y\}$ or $\{y/x\}$
$P(g(y), z)$	$\{x/g(y), z/g(g(y))\}$

but not with $P(Socrates, f(Socrates))$ or $P(g(y), y)$

Let us try unifying this expression, *P*(*x*, *g*(*x*)) with *P*(*x*, *y*). In this case, the first argument is the same. So, we move on, and we have a variable *y* and a term *g*(*x*), so we can replace *y* with *g*(*x*), so it becomes {*y* / *g*(*x*)}. Then the resulting expression would be *P*(*x*, *g*(*x*)).

Examples: Unification

$P(x, g(x))$ will unify with:

Expression	Necessary Substitution
$P(x, y)$	$\{y/g(x)\}$
$P(z, g(z))$	$\{z/x\}$ or $\{x/z\}$
$P(Socrates, g(Socrates))$	$\{x/Socrates\}$
$P(x, g(y))$	$\{x/y\}$ or $\{y/x\}$
$P(g(y), z)$	$\{x/g(y), z/g(g(y))\}$

but not with $P(Socrates, f(Socrates))$ or $P(g(y), y)$

$\theta = \{y/g(x)\}$

$P(x, g(x))\,\theta = P(x, g(x))$

$P(x, y)\,\theta = P(x, g(x))$

To explicitly show this example, here is $P(x, g(x))$, and here is $P(x, y)$, and we apply this substitution to both of these two expressions. We can think of this as E_1 and this as E_2. Then because there is no y in it, applying this substitution has no effect, and we get $P(x, g(x))$ itself. Whereas applying the theta on $P(x, y)$, we get $P(x, g(x))$ because y is now replaced with $g(x)$.

As a result, when we apply these two substitutions to the two expressions, we get the same form. So, we have unified these two expressions using this substitution, and hence this is the unifier for these two expressions.

Examples: Unification

$P(x, g(x))$ will unify with:

Expression	Necessary Substitution
$P(x, y)$	$\{y/g(x)\}$
$P(z, g(z))$	$\{z/x\}$ or $\{x/z\}$
$P(Socrates, g(Socrates))$	$\{x/Socrates\}$
$P(x, g(y))$	$\{x/y\}$ or $\{y/x\}$
$P(g(y), z)$	$\{x/g(y), z/g(g(y))\}$

but not with $P(Socrates, f(Socrates))$ or $P(g(y), y)$

Likewise, when we want to unify $P(x, g(x))$ and $P(z, g(z))$, initially, x and z are different, so we want to replace either x with z or z with x, and as a result, these two would now become the same. So, as we can see, it is not unique. It can either be this or that, but this is only the case because both

of these are variables.

Next, we want to unify $P(x, g(x))$ with $P(Socrates, g(Socrates))$, in this case, the first case, variable x versus $Socrates$. So, we have to replace x with $Socrates$. And once we do that, we can see that again, they can be unified.

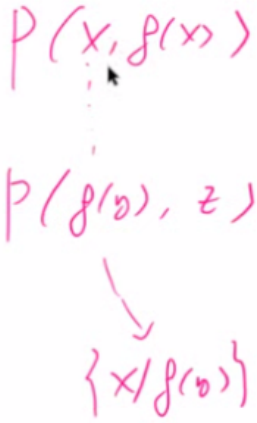
Same for $P(x, g(y))$, and $P(g(y), z)$. This is an interesting example, so let us take a look at this.

Examples: Unification

$P(x, g(x))$ will unify with:

Expression	Necessary Substitution
$P(x, y)$	$\{y/g(x)\}$
$P(z, g(z))$	$\{z/x\}$ or $\{x/z\}$
$P(Socrates, g(Socrates))$	$\{x/Socrates\}$
$P(x, g(y))$	$\{x/y\}$ or $\{y/x\}$
$P(g(y), z)$	$\{x/g(y), z/g(g(y))\}$

but not with $P(Socrates, f(Socrates))$ or $P(g(y), y)$



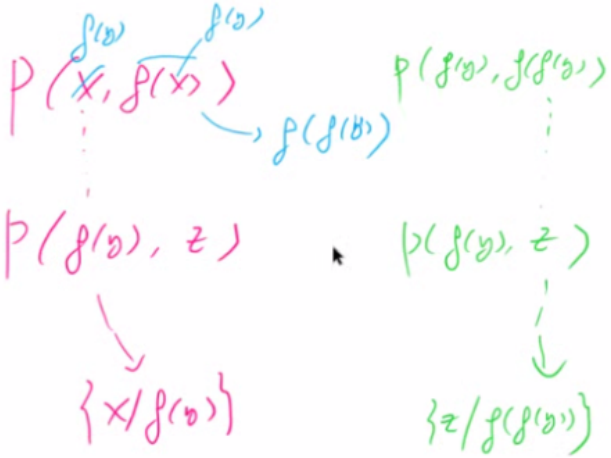
For us to resolve this difference between x and $g(y)$, we have to replace x with $g(y)$. And as a result, what we get is this will become $g(y)$, and x here will become $g(y)$. So, this whole thing would now become $g(g(y))$.

Examples: Unification

$P(x, g(x))$ will unify with:

Expression	Necessary Substitution
$P(x, y)$	$\{y/g(x)\}$
$P(z, g(z))$	$\{z/x\}$ or $\{x/z\}$
$P(Socrates, g(Socrates))$	$\{x/Socrates\}$
$P(x, g(y))$	$\{x/y\}$ or $\{y/x\}$
$P(g(y), z)$	$\{x/g(y), z/g(g(y))\}$

but not with $P(Socrates, f(Socrates))$ or $P(g(y), y)$



As a result of this, what we get is $P(g(y), g(g(y)))$, which is the first expression where we apply this substitution. The second expression would remain the same because it did not include x in its arguments. And now the difference is between z and $g(g(y))$. So, by simply replacing z with $g(g(y))$, we can get the unified expression, which would be this one here.

Examples: Unification

$P(x, g(x))$ will unify with:

Expression	Necessary Substitution
$P(x, y)$	$\{y/g(x)\}$
$P(z, g(z))$	$\{z/x\}$ or $\{x/z\}$
$P(Socrates, g(Socrates))$	$\{x/Socrates\}$
$P(x, g(y))$	$\{x/y\}$ or $\{y/x\}$
$P(g(y), z)$	$\{x/g(y), z/g(g(y))\}$

but not with $P(Socrates, f(Socrates))$ or $P(g(y), y)$

Of course, it is not always possible to unify two expressions. For example, if we consider $P(x, g(x))$ and $P(Socrates, f(Socrates))$, then let us say we replace this x with $Socrates$, then now we have $g(x)$ or $g(Socrates)$ versus $f(Socrates)$ because these two cannot be resolved in any way because there are only terms in this case, or in this case, functions or there is no variable in this pair. There is nothing to replace, and we are stuck. And the same for this.

Disagreement Set

Let W be a nonempty set of expressions $\{E_1, ..., E_n\}$. The **disagreement set** D of W is obtained by locating the first symbol (counting from the left) at which not all the expressions in W have exactly the same symbol, and then extracting from each expression E_i in W the subexpression that begins with the symbol occupying that position.

Example:

$$W = \{P(x, y, a, \mid f(x)),\ P(x, y, a, \mid g(x)),\ P(x, y, a, \mid z)\}$$

Symbols to the right of the vertical bar differ.

$D = \{f(x), g(x), z\}$

One last concept we need to know to write an algorithm for unification is the concept of a disagreement set. Let *W* be a non-empty set of expressions, E_1 to E_n. The disagreement set *D* of *W,* the set of expressions, is obtained by locating the first symbol, counting from the left, at which not all the expressions in *W* have exactly the same symbol, and then extracting from this expression E_i in *W* the subexpression that begins with the symbol occupying that position.

For example, let us say we have three expressions, *P*(*x*, *y*, *a*, *f*(*x*)). The second expression is *P*(*x*, *y*, *a*, *g*(*x*)). The third expression is *P*(*x*, *y*, *a*, *z*). So, start from the left and move one argument by one argument. And by the time we reach here, there is a disagreement.

So, in the first expression, there is *f*(). In the second expression, there is *g*(). In the last expression, there is *z*.

Disagreement Set

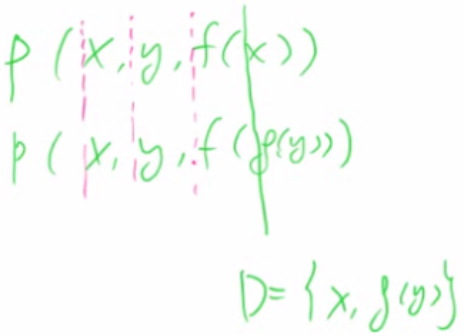
Let W be a nonempty set of expressions $\{E_1, \ldots, E_n\}$. The **disagreement set** D of W is obtained by locating the first symbol (counting from the left) at which not all the expressions in W have exactly the same symbol, and then extracting from each expression E_i in W the subexpression that begins with the symbol occupying that position.

Example:

$$W = \{P(x, y, a, \mid f(x)),\ P(x, y, a, \mid g(x)),\ P(x, y, a, \mid z)\}$$

Symbols to the right of the vertical bar differ.

$D = \{f(x), g(x), z\}$



We must be careful when finding the disagreement set because, in many cases, this disagreement set does not occur at the argument boundary. So, here is actually at this argument boundary. So, the first argument would have this disagreement set. In contrast, take a look at this example. So, here are the first argument, the second argument, and the third argument. The beginning of the third argument is the same, but the disagreement set is not *f*(*x*) and *f*(*g*(*y*)), but rather *x* and *g*(*y*) because we will have to go inside these functions to find where exactly the symbols do not agree.

Disagreement Set: More Examples

Examples:

1. $W = \{P(a), P(x)\}$ $\quad D = \{a, x\}$
2. $W = \{P(x, \underline{f(y, a)}), P(x, \underline{a}), P(x, \underline{g(h(k(x)))})\}$
   $D = \{f(y, a), a, g(h(k(x)))\}$
3. $W = \{P(x, f(g(\underline{h(y)}))), P(x, f(g(\underline{z})))\}$
   $D = \{h(y), z\}$

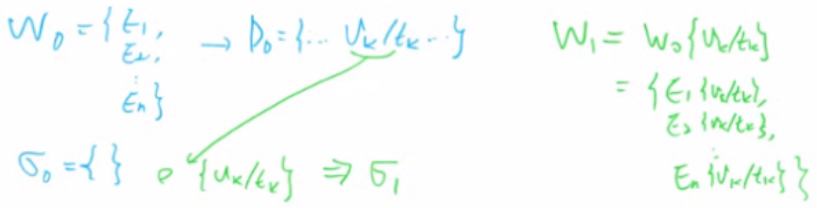
Let us look at some examples. When we have two expressions, *P*(*a*) and *P*(*x*), the disagreement set is {*a*, *x*}. If we have three expressions, *P*(*x*, *f*(*y*, *a*)), *P*(*x*, *a*), and *P*(*x*, *g*(*h*(*k*(*x*)))), then the disagreement set contains these three terms. So, the first is a function, the second is a constant, and the third is also a function. So, we have these three.

As I mentioned in the previous slide, in the end, when we have these nested functions, then we have to actually go inside the nested function to find the precise location where there is a disagreement. So, even though these two are the second argument of the predicate *P*, the disagreement set is not *f*(*g*(*h*(*y*))) and *f*(*g*(*z*)), but rather is *h*(*y*) and *z*.

Unification Algorithm

Let $W = \{E_1, ..., E_n\}$ be the set of expressions to be unified.

1. If necessary, **rename** variables so that no pair (E_i, E_j) from different clauses has any variables in common.
2. **Set** $k = 0, W_k = W, \sigma_k = \epsilon$ (empty substitution).
3. **If** W_k **is a singleton** (contains only one expr), stop; σ_k is a most general unifier for W. **Otherwise**, let D_k be the disagreement set for W_k.
4. **If** there exist elements v_k and t_k in D_k such that v_k **is a variable that does not occur in term** t_k, go to step 5. **Otherwise**, stop; W is not unifiable.
5. Let $\sigma_{k+1} = \sigma_k \circ \{v_k/t_k\}$ and $W_{k+1} = W_k\{v_k/t_k\}$. (Note that $W_{k+1} = W_k\sigma_{k+1}$)
6. Set $k = k + 1$ and go to step 3.



Finally, here is the full unification algorithm. So, we start with a set of expressions, E_1 to E_n. If necessary, we rename all of the variables in these expressions so there is no redundancy. To begin, we set the index k to zero and set the initial set W_0 to be the same set of expressions.

So, we start with W_0 equals the original expressions. And we set the initial unifier σ_0, because k is zero, it will be σ_0, to be an empty substitution.

If this set is a singleton, meaning that there is only a single expression in it, then we are done, and we return σ_k at that time as the most general unifier for the expression W. Otherwise, we find the disagreement set of all of these expressions. Again, we use the same approach we saw in the previous slide to get the disagreement set, and let us say at some point, we have this disagreement set where there is v_k / t_k in all of these.

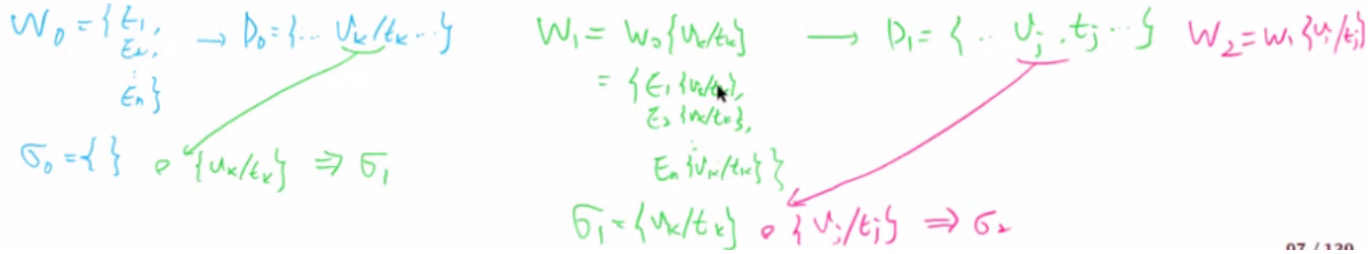
If we are able to find one variable and one term in this disagreement set, then we make that into a substitution and compose that with the existing unifier. So, in the first step, since the initial unifier is an empty substitution, the next step, σ_1 would be v_k / t_k.

Then what we do is we apply this substitution to the previous set of expressions. By doing this, we are simply applying this substitution to each expression in the previous set of expressions, and basically, we repeat this whole process.

Unification Algorithm

Let $W = \{E_1, ..., E_n\}$ be the set of expressions to be unified.

1. If necessary, **rename** variables so that no pair (E_i, E_j) from different clauses has any variables in common.
2. **Set** $k = 0, W_k = W, \sigma_k = \epsilon$ (empty substitution).
3. **If** W_k **is a singleton** (contains only one expr), stop; σ_k is a most general unifier for W. **Otherwise**, let D_k be the disagreement set for W_k.
4. **If** there exist elements v_k and t_k in D_k such that v_k **is a variable that does not occur in term** t_k, go to step 5. **Otherwise**, stop; W is not unifiable.
5. Let $\sigma_{k+1} = \sigma_k \circ \{v_k/t_k\}$ and $W_{k+1} = W_k\{v_k/t_k\}$. (Note that $W_{k+1} = W_k\sigma_{k+1}$)
6. Set $k = k + 1$ and go to step 3.



For example, once we have the disagreement set from this resulting set of expressions, and we

find v_j and t_j in it, then we use that to compose with the previous σ_1 to derive σ_2. Again, apply this new substitution to all the expressions in W_1 to get W_2, and we repeat that.

Finally, by repeatedly applying this kind of substitution to the set of expressions, if we finally get a single expression, there is the singleton, then we are done. And the σ_k at that time is the most general unifier.

Unification Theorem

If W is a finite nonempty *unifiable* set of expressions, then the unification algorithm will always terminate at step 3, and the last σ_k is a most general unifier for W (i.e. not unnecessary substitutions). The algorithm must terminate because each pass through the loop reduces the number of variables by 1, and there are only finitely many of them.

The unification theorem says if W, the set of expressions, is a finite non-empty, and unifiable set of expressions, then the unification algorithm in the previous slide will always terminate at step 3, which is where there is the singleton that is detected, and the last σ_k is the most general unifier for the initial set of expressions, W.

The algorithm must terminate because each pass through the loop reduces the number of variables by one, and there are only finitely many of them.

Unification Example

$P(x, f(x), z)$ vs.
$P(g(y), f(g(a)), y)$:

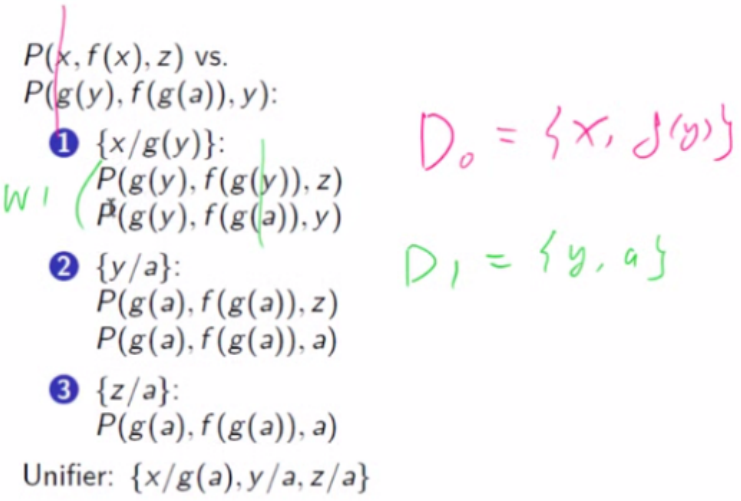
1. $\{x/g(y)\}$:
   $P(g(y), f(g(y)), z)$
   $P(g(y), f(g(a)), y)$
2. $\{y/a\}$:
   $P(g(a), f(g(a)), z)$
   $P(g(a), f(g(a)), a)$
3. $\{z/a\}$:
   $P(g(a), f(g(a)), a)$

Unifier: $\{x/g(a), y/a, z/a\}$

$D_0 = \{x, g(y)\}$

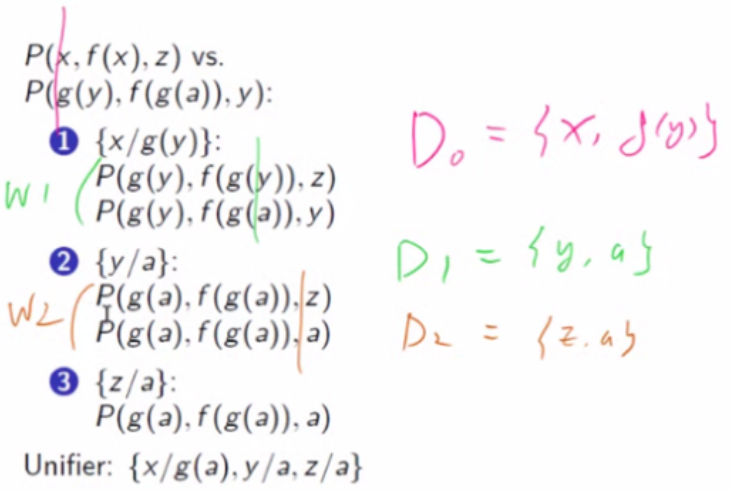
Let us look at this example. The first expression is $P(x, f(x), z)$. The second expression is $P(g(y), f(g(a)), y)$. From here, we can see that the disagreements, that D_0 is $\{x, g(y)\}$, and since this is a variable and that is a term, we can use this substitution. And when we apply this to these two expressions, then we get w_1.

Unification Example

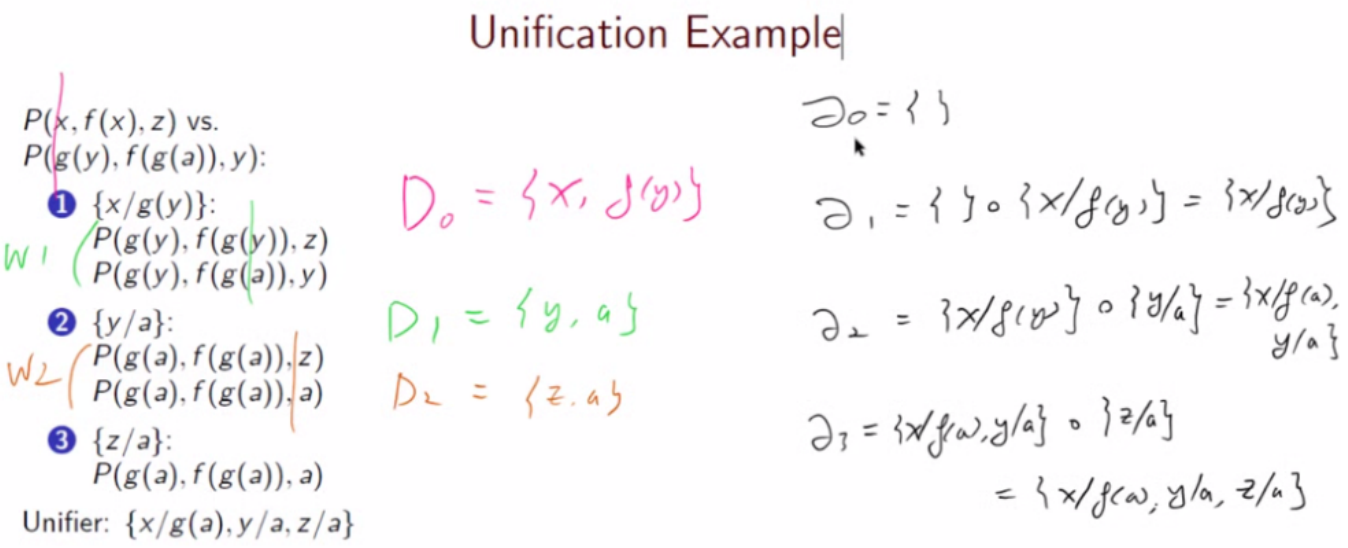


Now we have W_1 with these two expressions, and again, we swipe from the left to the right to identify the disagreement set. So, D_1 now becomes $\{y, a\}$, and from that, we can derive the substitution $\{y / a\}$, then apply that to W_1. And as a result, we get W_2.

Unification Example



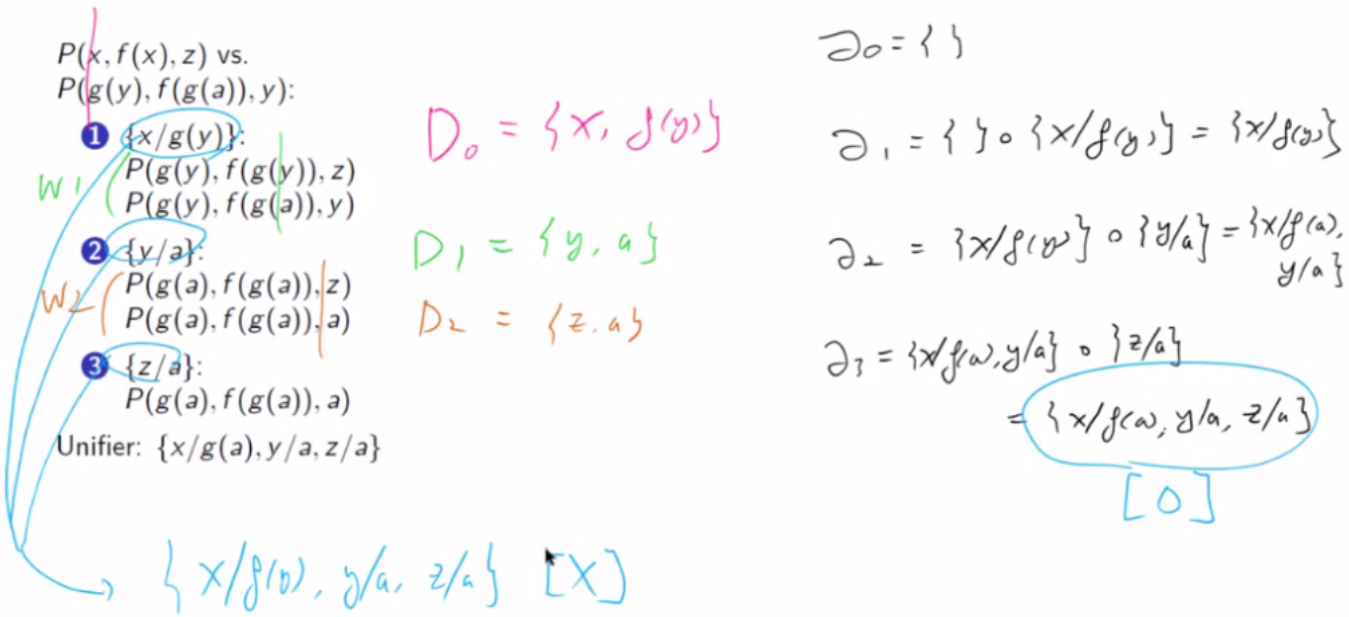
Again, we go from left to right and discover the disagreements between these two expressions. We end up with D_2 = {*z*, *a*}. Again, we can use this as a substitution, and then finally, when we apply this substitution to these two expressions, we end up with a single expression. That is a singleton, and we are done.



As for the unifier, initially, it is empty. After composing this empty substitution, the unifier, with {*x* / *g*(*y*)}, (which is that), then we get σ_1 is {*x* / *g*(*y*)}. In the next step, σ_2 is σ_1, which is {*x* / *g*(*y*)}, composed with {*y* / *a*}, which is that. Then *y* becomes *a*, and we add {*y* / *a*} to the end. So, we have two substitutions, {*x* / *g*(*a*), *y* / *a*}.

Finally, σ_3 is a composition of σ_2 and {*z* / *a*}. So, we compose {*x* / *g*(*a*), *y* / *a*} with {*z* / *a*}. In this case, *z* does not appear in the first substitution, so it becomes {*x* / *g*(*a*), *y* / *a*, *z* / *a*}, which is the unifier.

Unification Example



We have to be careful because if we just pull these three substitutions and instead of composing them, if we just collect them, then this will not be the correct unifier. So, we can see that here instead of *y*, we have to have *a*. So, this correct unifier is, of course, different from this incorrect unifier because, in this case, we did not go through the composition process.

Unification Example

$P(x, f(x), z)$ vs.
$P(g(y), f(g(a)), y)$:

1. $\{x/g(y)\}$:
   $P(g(y), f(g(y)), z)$
   $P(g(y), f(g(a)), y)$
2. $\{y/a\}$:
   $P(g(a), f(g(a)), z)$
   $P(g(a), f(g(a)), a)$
3. $\{z/a\}$:
   $P(g(a), f(g(a)), a)$

Unifier: $\{x/g(a), y/a, z/a\}$

With everything we covered so far, including standard forms, substitutions, and unification, we are now ready to go on with a full-blown resolution theorem proving, which is the topic of the following

lectures.

CSCE 625 **MODULE 9** | RESOLUTION: UNIFICATION